



C Programming

Presenter

Usha Rani Sargurunathan

Project Engineer

Software Technology Group

Type Casting in C

Type Casting in C

Type casting in C is the process of converting a variable or expression from one data type to another.

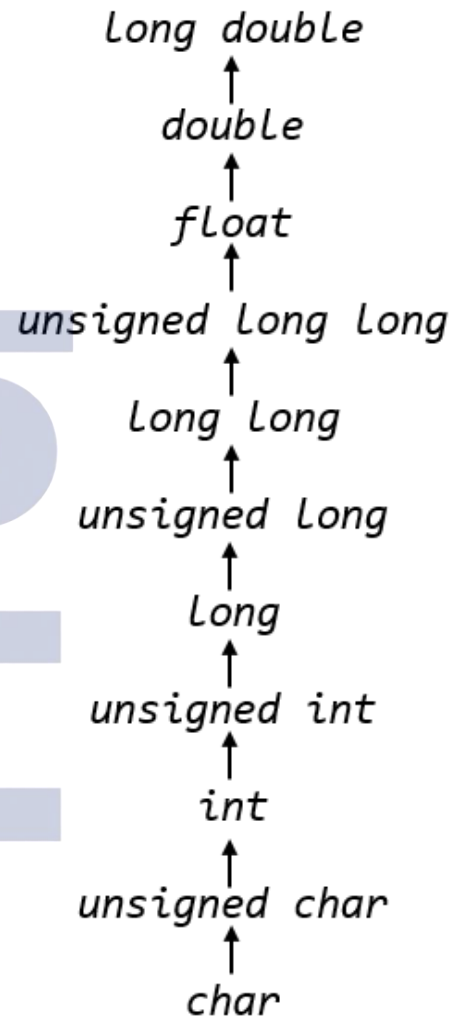
Syntax:

```
int x;
```

```
float y;
```

```
y = (float) x;
```

In type conversion, the **destination data type can't be smaller than the source data type**.



Type Casting

```
graph TD; A[Type Casting] --> B[Implicit Type Casting]; A --> C[Explicit Type Casting];
```

Implicit Type
Casting

Explicit Type
Casting

Implicit Type Casting

- When the compiler automatically transforms the data from one type to another, it is referred to as implicit casting or automated type conversion.
- All the data types of the variables are upgraded to the data type of the variable with the largest data type to avoid loss of data.
- bool -> char -> short int -> int -> unsigned int -> long -> unsigned long -> long -> float -> double -> long double

```
#include <stdio.h>
```

```
int main() {
```

```
int num1 = 10;
```

```
float num2 = 5.5;
```

```
float result;
```

```
result = num1 + num2; // Implicit cast from int  
to float
```

```
printf("The result is: %f\n", result);
```

```
return 0;
```

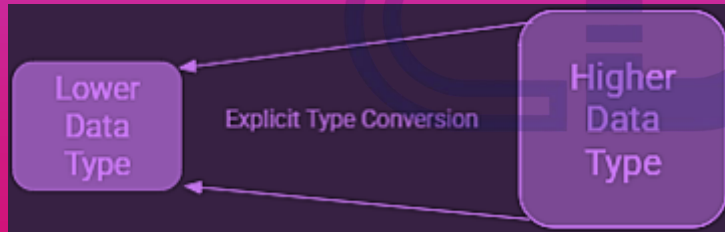
```
}
```

Output:

The result is: 15.500000

Explicit Type Casting

- Using the cast operator, explicit casting entails explicitly changing one data type to another.
- It needs explicit instructions in the code and allows the programmer control over type conversion.
- Syntax : (type) expression



```
#include <stdio.h>
```

```
int main() {
```

```
float num1 = 15.6;
```

```
int num2;
```

```
num2 = (int) num1; // Explicit cast from float  
to int
```

```
printf("The result is: %d\n", num2);
```

```
return 0;
```

```
}
```

Output:

The result is: 15

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a, b;
```

```
    float c;
```

```
    a=5;
```

```
    b=2;
```

```
    c=a/b;
```

```
    printf("C is %f", c);
```

```
}
```

Activity

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a, b;
```

```
    float c;
```

```
    a=5;
```

```
    b=2;
```

```
    c=(float)a/b;
```

```
    printf("C is %f", c);
```

```
}
```

Answer

Unary operators in C

Unary operators are the operators that perform operations on a single operand to produce a new value.

01	Unary minus (-)
02	Increment (++)
03	Decrement (--)
04	NOT (!)
05	Addressof operator (&)
06	sizeof()

1. Unary Minus

- The minus operator (`-`) **changes the sign** of its argument.
- A positive number becomes negative, and a negative number becomes positive.

```
int a = 10;
```

```
int b = -a; // b = -10
```

- Unary minus is different from the subtraction operator, as subtraction requires two operands.

Implementation of Unary Minus Operator

```
#include <stdio.h>

int main()
{
    int positiveInteger = 100; // declaring a positive integer
    int negativeInteger = - positiveInteger;

    // using - sign to make the value of positive integers to negative

    printf("Positive Integer = %d\n", positiveInteger);
    printf("Negative Integer = %d", negativeInteger);

    return 0;
}
```

Output:

Positive Integer = 100

Negative Integer = -100

2. Increment

The increment operator (++) is used to increment the value of the variable by 1.

Prefix increment

Example: In this method, the operator precedes the operand (e.g., ++a). The value of the operand will be altered before it is used.

```
int a = 1;
```

```
int b = ++a; // b = 2
```

Postfix increment

In this method, the operator follows the operand (e.g., a++). The value operand will be altered after it is used.

Example:

```
int a = 1;
```

```
int b = a++; // b = 1
```

```
int c = a;   // c = 2
```

Implementation of the increment (++)

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a = 5;
```

```
    int b = 5;
```

```
    printf("Pre-Incrementing a = %d\n", ++a);
```

```
    printf("Post-Incrementing b = %d", b++);
```

```
    return 0;
```

```
}
```

Output :

Pre-Incrementing a = 6

Post-Incrementing b = 5

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
int a, b, c;
```

```
a=6;
```

```
b=8;
```

```
c=++a + b++;
```

```
printf("C is %d", c);
```

```
}
```

Activity

```
#include<stdio.h>

int main()
{
    int a, b, c;
    a=6;
    b=8;
    c=++a + b++;
    printf("C is %d", c);
}
```

Answer

Output :

C is 15

3. Decrement

The decrement operator (`--`) is used to decrement the value of the variable by 1.

Prefix decrement

In this method, the operator precedes the operand (e.g., `--a`). The value of the operand will be altered before it is used.

Example:

```
int a = 1;  
  
int b = --a; // b = 0
```

Postfix decrement

In this method, the operator follows the operand (e.g., `a--`). The value of the operand will be altered after it is used.

Example:

```
int a = 1;  
  
int b = a--; // b = 1  
  
int c = a;   // c = 0
```

Implementation of the decrement (--):

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a = 5;
```

```
    int b = 5;
```

```
    printf("Pre-Decrementing a = %d\n", --a);
```

```
    printf("Post-Decrementing b = %d", b--);
```

```
    return 0;
```

```
}
```

Output :

Pre-Decrementing a = 4

Post-Decrementing b = 5

4. NOT (!)

⇒ The logical NOT operator (!) is used to reverse the logical state of its operand. If a condition is true, then the Logical NOT operator will make it false.

⇒ Example:

If x is true, then !x is false

If x is false, then !x is true

```
#include <stdio.h>

int main()
{
    int a = 10;
    int b = 5;

    if (!(a > b))
        printf("b is greater than a\n");
    else
        printf("a is greater than b");

    return 0;
}
```

Output:

a is greater than b

5. Addressof operator (&)

The addressof operator (&) gives an address of a variable. It is used to return the **memory address of a variable**. These addresses returned by the address-of operator are known as **pointers** because they “point” to the variable in memory.

Example:

& gives an address on variable n

```
int a;
```

```
int *ptr;
```

```
ptr = &a; // address of a is copied to the location ptr.
```

Implementation of the Addressof operator(&)

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a = 20;
```

```
    printf("Address of a = %p", &a);
```

```
    return 0;
```

```
}
```

Output

Address of a = 0000004a79ffa3c

6. sizeof()

- This operator returns the size of its operand, in bytes. The sizeof() operator always precedes its operand. The operand is an expression, or it may be a cast.
- Implementation of sizeof() operator :

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
// printing the size of double and int using sizeof
```

```
printf("Size of double: %d\n", sizeof(double));
```

```
printf("Size of int: %d\n", sizeof(int));
```

```
return 0;
```

```
}
```

Output

Size of double: 8

Size of int: 4

Relational and Logical operators

Relational and logical operators

Relational and logical operators result in 0 or 1.

0 – indicate false condition

1 – indicate true condition

Relational operators

<, >, <=, >=, ==, !=

Logical operators

&&, ||, !

Relational Operator

Relational operators in C are defined to perform comparison of two values.

Operator	Description	Example
==	Checks if the values of two operands are equal or not. If yes, then the condition becomes true.	(A == B)
!=	Checks if the values of two operands are equal or not. If the values are not equal, then the condition becomes true.	(A != B)
>	Checks if the value of left operand is greater than the value of right operand. If yes, then the condition becomes true.	(A > B)
<	Checks if the value of left operand is less than the value of right operand. If yes, then the condition becomes true.	(A < B)
>=	Checks if the value of left operand is greater than or equal to the value of right operand. If yes, then the condition becomes true.	(A >= B)
<=	Checks if the value of left operand is less than or equal to the value of right operand. If yes, then the condition becomes true.	(A <= B)

Logical Operator

Logical operators in C evaluate to either True or False.

P	Q	P && Q	P Q	! P
T	T	T	T	F
T	F	F	T	F
F	T	F	T	T
F	F	F	F	T

Operator	Description	Example
&&	Called Logical AND operator. If both the operands are non-zero, then the condition becomes true.	(A && B)
	Called Logical OR Operator. If any of the two operands is non-zero, then the condition becomes true.	(A B)
!	Called Logical NOT Operator. It is used to reverse the logical state of its operand. If a condition is true, then Logical NOT operator will make it false.	!(A)

Bit-wise operators

- Bitwise operators in C allow low-level manipulation of data stored in computer's memory.
- The C language provides six operators for bit manipulation they operate on the individual bits of the operands.
- The Bitwise operators available in C are

Operator	Description	Example
&	Binary AND Operator copies a bit to the result if it exists in both operands.	(A & B)
	Binary OR Operator copies a bit if it exists in either operand.	(A B)
^	Binary XOR Operator copies the bit if it is set in one operand but not both.	(A ^ B)
<<	Binary Left Shift Operator. The left operands value is moved left by the number of bits specified by the right operand.	A << 2
>>	Binary Right Shift Operator. The left operands value is moved right by the number of bits specified by the right operand.	A >> 2
~	Bitwise NOT operator results in one's complement of its operand.	(~A)

Bitwise AND Operator (&) in C

The bitwise AND (&) operator performs as per the following truth table

bit a	bit b	a & b
0	0	0
0	1	0
1	0	0
1	1	1

Example : - 10 & 5

0000 1010 -> Binary of 10

0000 0101 -> Binary of 5

0000 0000 O/P is 0

Bitwise OR Operator (|) in C

The bitwise OR (|) operator performs as per the following truth table –

bit a	bit b	a b
0	0	0
0	1	1
1	0	1
1	1	1

Example : - 10 & 5

0000 1010 -> Binary of 10

0000 0101 -> Binary of 5

0000 1111 O/P is 15

Bitwise XOR Operator (^) in C

The bitwise XOR (^) operator performs as per the following truth table –

bit a	bit b	a ^ b
0	0	0
0	1	1
1	0	1
1	1	0

Example : - 10 & 5

0000 1010 -> Binary of 10

0000 0101 -> Binary of 5

0000 1111 O/P is 15

Bitwise NOT Operator (~) in C

The bitwise NOT (~) operator performs as per the following truth table and results in one's complement of its operand -

a	~a
0	1
1	0

Example : - ~61

~ 0011 1100 -> Binary of 60

1100 0011 O/P is -61 -> operation results in -61 in 2's complement form

Left Shift << and Right shift >>

The bitshift operators take two arguments, and looks like:

- $x \ll n$: shifts the value of x left by n bits
- $x \gg n$: shifts the value of x right by n bits

Left shift operator :

$\text{num} \ll n = \text{num} * 2^{\text{raise to } n}$

$5 \ll 2 = 5 * 2^{\text{to the power } 2}$

$5 * 4 = 20$

Original: 00000101 -> binary for 5

Shifted: 0000100 -> binary for 10

(after left shifting by 2)

Right Shift Operator :

$\text{num} \gg n = \text{num} / 2^{\text{raise to } n}$

$9 \gg 1 = 9 / 2^{\text{to the power } 1} = 4$

Original: 00001001 -> binary for 9

Shifted: 00000100 -> binary for 4

(after right shifting by 1)

Find the output for $8 \gg 1$ and $12 \ll 1$

Activity

The output for $8 \gg 1$: 4 and $12 \ll 1$: 24

Answer

THANK YOU